

Backend Pipeline + Authentication Flow + Logging + Middleware Understanding

1. API Request Pipeline Workflow

```
Client Request
↓
ExceptionHandlerMiddleware (wraps everything in try-catch)
↓
CorrelationIdMiddleware (assigns unique request ID)
↓
RequestLoggingMiddleware (logs request + response)
↓
Authentication (JWT validation)
↓
Authorization (role/policy check)
↓
Controller (entry point)
↓
MediatR → Handler → DB / Services
↓
Response returned
↓
LoggingMiddleware logs response
↓
ExceptionMiddleware catches failures (if any)
```

2. MIDDLEWARES

1. ExceptionHandlingMiddleware

Purpose:

- Global error handler for entire application

Key Behavior:

- Does NOT process request immediately
- Wraps entire pipeline using try-catch

Flow:

try

→ await _next(context) (let request flow)

catch

→ intercept exceptions

→ map to HTTP status codes

→ return standardized ApiResponse

- Centralized error handling
- No try-catch needed in controllers/handlers
- Ensures consistent API response format

2. CorrelationIdMiddleware

Purpose:

- Assign unique ID to each request for tracing

Flow:

Request comes

- Check header for existing ID
- Else generate GUID
- Store in HttpContext.Items (request memory)
- Add to response header
- Pass to next middleware

HttpContext.Items = request-scoped storage

Used so:

- All middlewares
- Controllers
- Logs

can access SAME correlation ID

3. RequestLoggingMiddleware

Purpose:

- Track full lifecycle of request

Flow:

Before _next:

- Log incoming request

Call _next(context)

- executes entire pipeline

After _next:

- Log response + execution time

On Exception:

- Log failure
- rethrow → handled by ExceptionMiddleware

Logging BEFORE + AFTER gives:

- full trace
- performance timing
- debugging visibility

3. AUTHENTICATION & AUTHORIZATION

AuthExtensions.cs

builder.Services.AddAuth(configuration);

1. Authentication (JWT)

- Reads token from header:

Authorization: Bearer <token>

- Validates:

→ Issuer

→ Audience

→ Expiry

→ Signature (Secret key)

- Extracts Claims:

→ UserId

→ Email

→ Role

2. Authorization (Policies)

AdminOnly → requires Role = Admin

UserOnly → requires Role = User

Authentication = WHO are you?

Authorization = WHAT can you access?

4. API LOGIN FLOW

Client sends Email + Password

↓

AuthController

↓

LoginCommand (via MediatR)

↓

LoginCommandHandler

→ Fetch user from DB

→ Verify password (BCrypt)

→ Generate JWT token

↓

JwtTokenService

→ Create claims

→ Sign token

↓

Return LoginResponse (Token)

↓

Wrapped in ApiResponse

↓

Sent to client

1. AuthController

Role:

- Entry point

- Converts request → command
- Delegates logic to MediatR
- Returns response

2. LoginCommandHandler

Role:

- Core business logic

Steps:

1. Get user from DB
2. Validate password
3. Generate token
4. Return response

3. JwtTokenService

Role:

- Create JWT token

Steps:

1. Build claims (Id, Email, Role)
2. Create signing key from Secret
3. Sign token
4. Add expiry, issuer, audience
5. Return token string

Controller → no logic

Handler → business logic

Service → infrastructure (token creation)

4. API RESPONSE STANDARDIZATION

Purpose:

- Consistent API response format

Structure:

Success

Message

Errors

Data

Every response looks same:

- success
- failure
- validation
- errors

5. LOGGING SYSTEM

```
graph TD; A["[Correlation] Assigned ID"] --> B["[Middleware] Incoming Request"]; B --> C["[Controller] Login endpoint hit"]; C --> D["[MediatR] Handling LoginCommand"]; D --> E["[Handler] Fetching user"]; E --> F["[Handler] Verifying password"]; F --> G["[Handler] Generating token"]; G --> H["[Controller] Returning response"]; H --> I["[Middleware] Response logged with time"];
```

[Correlation] Assigned ID
↓
[Middleware] Incoming Request
↓
[Controller] Login endpoint hit
↓
[MediatR] Handling LoginCommand
↓
[Handler] Fetching user
↓
[Handler] Verifying password
↓
[Handler] Generating token
↓
[Controller] Returning response
↓
[Middleware] Response logged with time

Logs confirm REAL execution flow

→ matches architecture perfectly